Project Report

By:

Ritvik Mahajan

25BCE10509

CSE Core

Introduction

This project is a small, practical tool—a Research AI agent built with Python and frameworks like langchain.

My main goal was to create something that helps users with their research-related tasks and provides them with real-time information.

It all runs directly in the terminal and uses specific tools like Wikipedia search and the DuckDuckGo search tool to retrieve real-time information on current world events and provide extensive information.

Problem Statement

My project tackles a problem related to research and information purposes. Users find it difficult to find the particular information they are looking for on the internet, but are not able to find it in time, or have to read long paragraphs just to find the information.

The solution?

A fast Research AI Agent that caters to the problem and provides the specific information the user is looking for by extensively searching using tools like Wikipedia for factual information and DuckDuckGo search for real-time information within seconds.

Functional Requirements

- ✓ **Multi-Source Data Retrieval:** The agent must be able to fetch real-time data from two distinct sources: Wikipedia (knowledge base) and DuckDuckGo (web news).

- ✓ **Structured Output Generation:** The system converts unstructured tool outputs into a predefined JSON/Pydantic format (Topic, Report, Sources, Tools Used).

- ✓ **Natural Language Processing:** The agent accurately interprets user queries to determine which specific tool (or combination of tools) is required for the task.

- ✓ **Session Management:** The agent maintains a conversation loop, allowing the user to ask follow-up questions or exit the program gracefully via a "quit" command.

Non-functional Requirements

- ✓ **Latency (Performance):** using Groq's Llama-3 or DeepSeek models, the agent provides a structured response within 3–5 seconds for standard queries to ensure a "real-time" experience.

- ✓ **Accuracy (Reliability):** The agent prioritizes groundedness—meaning it only reports facts found through the tools rather than hallucinatory professional details.
- ✓ **Scalability:** The architecture is modular enough to add new tools (e.g., Gmail or Slack) simply by updating the Tools.py file without rewriting the core logic.

- ✓ **Usability:** The Command Line Interface (CLI) provides clear feedback so the user can see which "thought process" the agent is currently following.

System Architecture: AI-Driven Professional Research Agent

The system follows a Decoupled Agentic Framework, utilizing the ReAct (Reason + Act) paradigm to bridge the gap between large language models and real-time external APIs.

1. Core Architectural Layers

The architecture is structured into four functional layers that separate reasoning from execution:

- User Interface Layer: A Command Line Interface (CLI) that captures natural language queries and renders the final structured JSON report.
- Reasoning Layer (The Brain): Powered by Groq (Llama-3.3). It analyzes user intent, determines the necessary tools, and synthesizes raw data into a coherent response.
- Orchestration Layer (The Controller): Built on LangChain's AgentExecutor. It manages the

iterative loop, maintains state, and handles the logic of calling tools in the correct sequence.

- Integration Layer (The Tools): Utilizes the Wikipedia and DuckDuckGo tools for web-scale data retrieval.

2. Functional Data Flow

The system operates through an autonomous four-step execution cycle:

1. Input & Analysis: The user provides a query, which the LLM decomposes into sub-tasks.
2. Tool Selection: The agent selects the appropriate tool (e.g., DuckDuckGo for current news).
3. Formatting: Pydantic Models validate the data, ensuring the final output adheres to a strict schema (Topic, Summary, Sources).

3. Technological Stack

Language: Python 3.10+

Orchestration: LangChain Framework

Inference Engine : Groq LPU (Llama-3)

Data Validation: Pydantic (Schema Enforcement)

4. Key Architectural Features

- Latency Optimization: Leveraging Groq's LPU inference for near-instantaneous reasoning.
- Scalability: Modular tool registration allowing for seamless addition of new APIs (e.g., Slack, Gmail).
- Reliability: Use of Grounded Generation to prevent hallucinations by forcing the LLM to cite tool-sourced data.

Design Decisions & Rationale

Decision	Why I Did It
Using Groq LPU	Minimal Latency & High-Speed Reasoning
ReAct Logic	Improved Accuracy & Self-Correction
Why Pydantic?	Schema Enforcement & Data Consistency

Implementation Details

The project is implemented in **Python 3.10+** using a modular **Agentic Framework** that bridges **Groq's Llama-3.3** models with real-time external APIs. The system's environment is secured via a decoupled .env configuration, while the **LangChain** orchestration layer manages a **ReAct (Reason + Act)** execution loop. For data retrieval, the agent utilizes a specialized toolset registry, including **DuckDuckGo** for web indexing and the Wikipedia search tool for factual information. A key technical feature is the implementation of **Pydantic BaseModels** and .with_structured_output(), which sanitizes raw API data into a consistent, machine-readable JSON format. By routing inference through **Groq's LPU architecture**, the system ensures low-latency, multi-step reasoning, providing a robust and scalable solution for professional automated research.

Screenshots/Results

1) Run the following command in the terminal

```
PS C:\Users\ritvi\OneDrive\Desktop\myfirstaiagent\Myfirstaiproject> python .\main.py

What can I help you research? (type 'quit' to exit):
```

2) Enter the query that you want to research or want information about.

```
What can I help you research? (type 'quit' to exit): "What are the latest developments in quantum computing as of 2026, and how do they relate to current cryptography standards?"
```

3) Click enter, and the AI agent responds with the topic, summary, tools, and sources used.

Topic: Post-Quantum Cryptography Standards

Research Summary

The National Institute of Standards and Technology (NIST) has finalized the first set of post-quantum cryptography (PQC) standards, which are designed to replace or augment classic public-key algorithms and are ready for immediate use. These standards include ML-KEM, ML-DSA, and SLH-DSA, which are designed to provide quantum-resistant cryptography for authentication and other applications. The development of PQC is necessary because practical quantum computers would break widely deployed public-key systems such as RSA, Diffie-Hellman, and ECC. Quantum computers use quantum particles called qubits that can be in two states at once, allowing them to perform certain calculations much faster than classical computers. This means that a quantum computer could potentially derive the private key of a cryptographic system, compromising its security. The migration to PQC is a present-day emergency, and organizations should start preparing for the transition to quantum-safe infrastructure. This includes updating libraries and protocols, ensuring crypto-agility, and building a comprehensive understanding of lattices and new key sizes. The use of post-quantum cryptography will provide a higher level of security against potential quantum computer attacks, and it is essential to stay ahead of the curve to ensure the long-term security of cryptographic systems.

Sources: National Institute of Standards and Technology (NIST)
Tools: NIST Website, Cryptography Research Papers

4) It then asks if you want to quit or not.

```
Do you want to continue? (Y/N): n
Thank you for using Research AI Assistant. Goodbye!
```

Challenges Faced

Building this agent pushed me beyond my existing knowledge base, necessitating a deep dive into real-time data retrieval and structured output parsing. Despite encountering frustrating technical roadblocks during the coding phase, a persistent approach to debugging and online collaborative learning allowed for their successful resolution. This experience proved invaluable in Understanding the nuances of modern AI development and the importance of resilience in project management.

Learnings & Key Takeaways

Learnings:

I learned how to create a GitHub account and repository, as well as how they function.

Additionally, I gained experience in writing a comprehensive report for the project, which will be beneficial for me in the future.

I developed my time management skills and learned to work efficiently under pressure.

Overall, I discovered how such projects can contribute to solving real-world problems through innovation and creativity.

Future Enhancements

If I were to expand this project, I would focus on the following areas:

1. **. Long-Term Memory & Personalization (RAG)**

- **Enhancement:** Integrating a **Vector Database** (like Pinecone or ChromaDB) to implement Retrieval-Augmented Generation (RAG).
- **Benefit:** The agent could remember a user's previous research on a specific company or person. If you ask about "the same CEO" next week, it won't need to burn API tokens re-searching basic facts, making it faster and more cost-effective.

2. Multi-Agent Orchestration

Instead of one "General" agent, I can move toward a **Team of Specialists** architecture.

- **Enhancement:** By using a framework like **LangGraph** or **CrewAI** to create separate agents:
  - *The Researcher:* Specializes in DuckDuckGo and Wikipedia.
  - *The Profile Analyst:* Deep-dives into LinkedIn data.
  - *The Editor:* Fact-checks the output and ensures professional formatting.
- **Benefit:** This reduces "cognitive load" on the LLM, leading to fewer hallucinations and more detailed reports.

3. Multimodal Research Capabilities

Most professional profiles and company news include charts, headshots, or product videos.

- **Enhancement:** Upgrading to a **Vision-capable LLM** (like Llama-3.2-Vision or Gemini 2.0).
- **Benefit:** The agent could "look" at a person's LinkedIn profile picture or a company's financial growth chart and include visual analysis in its final report, rather than just relying on text descriptions.

4. Automated Outreach & CRM Integration

- **Enhancement:** Use Composio to connect the agent to **Salesforce, HubSpot, or Gmail**.
- **Benefit:** After researching a lead, the agent could automatically create a "New Lead" entry in a CRM and draft a follow-up email based on the research findings, turning it into an end-to-end sales automation tool.